Personal Knowledge System

From Karpathy's LLM Wiki to a Practical Knowledge System

从 Karpathy 的 "LLM Wiki" 到一个可落地的个人知识管理系统

目录

1. 缘起：为什么做这个项目
2. 演进历程：从概念到落地的四步
3. 当前系统：完整的 Agentic Wiki
4. 关键设计决策
5. 项目现状
6. 未来方向

一、缘起：为什么做这个项目

1.1 知识管理的三大痛点

作为一名关注 AI 领域的产品经理，日常面临三个知识管理难题：

```
┌──────────────────────────────┐   ┌──────────────────────────────┐   ┌──────────────────────────────┐
│  信息过载                    │   │  碎片化存储                  │   │  检索效率低                  │
│                              │   │                              │   │                              │
│ 每天接触大量内容 │   │ 浏览器 / Notion │   │ 需要回顾时 │
│ 但读后即忘 │   │ 本地笔记 / 收藏 │   │ 要么翻找笔记 │
│ 无法形成积累 │   │ 散落在多个系统 │   │ 要么重新搜索 │
└──────────────────────────────┘   └──────────────────────────────┘   └──────────────────────────────┘
```

市面上所有知识管理工具的共性问题是：

> **只解决「存」，不解决「用」和「长」。**

1.2 Karpathy 的启发

2025 年，Andrej Karpathy 提出了 "LLM Wiki" 的概念。核心思想非常简单但颠覆：

> 不追求自动采集互联网内容，而是人工精选 Raw 材料，用 LLM 将 Raw 材料编译成结构化的 Wiki 词条，词条之间通过双向链接互联。随着时间推移，Wiki 会自动积累、生长，形成**知识复利**。

1.3 两种路线的本质区别

维度	传统 RAG	LLM Wiki（Agenti

内容来源	自动爬取，量大	人工精选，量小但精
存储方式	向量数据库（无结构）	结构化 Markdown + 双向链接
知识积累	线性堆积	网状复利（链接产生新关联）
查询方式	无状态语义检索	有状态 Wiki 导航
可维护性	难以人工干预	人工可阅读、可编辑、可审核
知识形态	黑盒向量	人类可读的词条网络

核心决策：选择 Agentic Wiki 路线 -- 让知识系统像维基百科一样有结构、可浏览、可维护，而不是一个黑盒向量库。

1.4 核心范式对比

```
传统 RAG                    Agentic Wiki

┌──────────────────────┐    ┌──────────────────────────────┐
│ 爬取所有内容 │    │ 人工精选 Raw 材料 │
│ ┌──────────────┐ │    │ ┌──────────────┐ │
│ │ 向量存储 │ │    │ │ LLM 编译 │ │
│ │ (黑盒) │ │    │ │ (可审核) │ │
│ └──────────────┘ │    │ └──────────────┘ │
│      │      │      │      │
│ 语义相似度检索 │    │ ┌──────▼──────┐ │
│ (无解释) │    │ │ Wiki 词条网络 │ │
└──────────────────────┘    │ │ (双向链接) │ │
                            │ └──────────────┘ │
                            │      │      │
                            │ ┌──────▼──────┐ │
                            │ │ 有状态查询 │ │
                            │ │ (知识导航) │ │
                            │ └──────────────┘ │
                            └──────────────────────────────┘
```

二、演进历程：从概念到落地的四步

阶段一：探索与试错 -- RAG 模式

项目最早期采用了传统 RAG 路线，构建了一个全自动化的知识采集系统：

```
RSS/爬虫采集 ── 优先级过滤 ── 摘要生成 ── 向量存储 ── 语义搜索/RAG问答
    │          │          │          │          │
    ▼          ▼          ▼          ▼          ▼
 50+信源    关键词打分   LLM摘要   FAISS+BM25   回答生成
 网页爬取   时效加成     分类标签   知识图谱     日报周报
 视频字幕   忽略词过滤   文件保存   NetworkX     APScheduler
```

包含的模块（共 13 个目录，50+ 文件）：

模块	文件数	功能
src/collector/	4	RSS 抓取、网页爬虫、视频字幕提取
src/retriever/	8	向量存储、BM25、语义搜索、RAG 问
src/processor/	4	摘要生成、优先级过滤、日报、周报
src/scheduler/	3	APScheduler 定时任务调度
src/graph/	3	SQLite + NetworkX 知识

src/agent/	3	旧 RAG API、Context In
src/mcp/	2	MCP Server 工具定义

发现的五个核心问题：

#	问题	影响
1	自动采集 = 信息垃圾场	RSS/爬虫灌入大量低质量噪声，知识库线
2	向量搜索是黑盒	用户不知道"为什么返回这个结果"，无法干
3	自动摘要质量不可控	LLM 自动摘要无法替代人工筛选后的精炼
4	自动化调度 = 成本浪费	定时任务持续消耗 API 额度，产出价值
5	没有知识复利	每篇文章孤立存储，没有链接产生新关联

关键结论：自动化采集这条路走不通。知识的价值不在于「量」，而在于「精选」和「关联」。

阶段二：架构重构 -- 转向 Agentic Wiki

基于阶段一的教训，进行了根本性的架构重构：

五项设计原则：

#	原则	说明
1	手动输入，不可变	Raw 材料由用户手动保存，系统不修改原
2	LLM 编译，可审核	Raw → Wiki 的过程透明，每个词
3	双向链接，网状生长	[[概念]] 语法互联，形成知识网络
4	index.md 导航	LLM 生成层级索引，人类 + LLM
5	手动触发	编译、查询、健康检查全由用户控制

新架构的数据流：

```
┌──────────────────────┐   ┌──────────────────────────────┐   ┌──────────────────────────────┐
│  raw/                │   │  compiler/                   │   │  wiki/                       │
│  Raw 材料            │──────│  LLM 编译器                  │──────│  Wiki 词条网络               │
│  (手动输入)          │   │  概念提取                    │   │  (结构化知识)                │
│                      │   │  双向链接                    │   │  index.md                    │
└──────────────────────┘   └──────────────────────────────┘   └──────────────────────────────┘
                                          │
                                ┌─────────────────────────▼─────────────────────────┐
                                │  query/                                           │
                                │  有状态深度查询                                   │
                                │  Token 预算管理                                   │
                                │  链接遍历                                         │
                                └───────────────────────────┬───────────────────────┘
                                          │
                                ┌─────────────────────────▼─────────────────────────┐
                                │  maintenance/                                     │
                                │  健康检查                                         │
                                │  矛盾/过期/断裂检测                               │
                                └───────────────────────────────────────────────────┘
```

阶段三：精简与聚焦 -- 做减法

完整的 Agentic Wiki 实现后，发现旧架构残留导致项目臃肿。进行了第二轮精简：

精简成果一览：

指标	精简前	精简后	减少
代码行数	9,400+ 行	~5,000 行	-47%
依赖包	42 行	16 行	-62%
Python 模块	13 个目录	5 个	-62%
脚本文件	26 个	7 个	-73%
配置文件	6 个	2 个	-67%
测试文件	9 个空壳	0 个	100%

删除了什么（整个模块）：

```
src/collector/    RSS / 爬虫 / 视频采集
src/retriever/    向量存储 / 语义搜索 / BM25
src/processor/    摘要 / 优先级 / 日报 / 周报
src/scheduler/    定时调度
src/graph/        旧知识图谱 (SQLite + NetworkX)
src/agent/        旧 RAG API / ContextInjector
src/mcp/          MCP Server
deploy/           plist daemon / 启动脚本
tests/            9 个空壳测试
```

保留并优化了什么：

```
src/compiler/     LLM 编译器 (Raw → Wiki)
src/query/        有状态 Wiki 查询引擎
src/maintenance/  健康检查
src/server/       Wiki Chat API
src/utils/        模型路由 (精简版)
英译中翻译        保留在 wiki_builder 中
PDF 转 Markdown   手动输入辅助工具
```

阶段四：CLI Skill 集成 -- 跨项目可用

为了让知识库能在 Claude Code 中跨项目使用，创建了 wiki-cli skill：

特性	说明
零 MCP 开销	直接 CLI 调用，不传输 JSON-R
跨项目可用	注册为全局 skill，任意项目目录下可
便捷脚本	wiki.sh 提供一键统计、查询、

```
# 在任何项目目录下使用
./wiki.sh query "什么是 Transformer？"
./wiki.sh compile --all
./wiki.sh stats
./wiki.sh health
```

三、当前系统：完整的 Agentic Wiki

3.1 核心工作流：四步闭环

```
┌──────────────────────────────────────┐
```

```
│ 第一步    │
│ 手动输入  │──────────────────────────────────────────────┐
│ Raw 材料  │                                              │
└──────────┬──────────┘                                    │
           │                                               ▼
┌──────────▼──────────┐   ┌──────────────────────┐   ┌──────────────────────┐
│ 第二步    │  │ 第四步    │   │ 第三步    │
│ LLM 编译  │────│ 健康检查  │────────────│ 深度查询  │
│ Wiki 词条 │  │ 维护清理  │   │ 知识复利  │
└──────────────────────┘   └──────────────────────┘   └──────────────────────┘
```

3.2 第一步：手动输入 Raw 材料

用户将自己筛选的有价值内容保存为 Markdown 文件：

```
---
type: article
source: https://example.com/article
tags: [LLM, Transformer]
status: raw
---

# 文章标题

[内容摘要或关键段落]

---

## 我的思考

[手写思考 / 个人理解] ← 这是知识区别于信息的核心
```

三种输入方式：

方式	场景	工具
手动创建 Markdown	最常用，直接粘贴内容	任意编辑器
PDF 转 Markdown	论文、报告等 PDF 文档	`scripts/pdf_to_md.p
英文内容直接粘贴	外文论文、技术博客	系统自动翻译编译

3.3 第二步：LLM 编译（Raw → Wiki）

```
PYTHONPATH=. python scripts/compile_raw.py --all
```

编译流程的 7 个步骤：

```
1. 读取 Raw 材料
   │
   ▼
2. 检测内容语言 ── 中文 ── 使用中文提取 prompt
   │          │
   │          └── 英文 ── 使用双语提取 prompt (含翻译)
   ▼
3. LLM 提取 3-8 个核心概念 (JSON 格式)
   │
   ▼
4. 为每个概念生成结构化词条
   │  ├── 定义（50 字以内）
```

```
    │   ├── 详细解释（200-300 字）
    │   ├── 关键点（3-5 条）
    │   └── 相关概念（双向链接）
    ▼
  5. 创建 Wiki 词条文件 (wiki/concepts/)
    │
    ▼
  6. 更新 wiki/index.md 导航（增量更新）
    │
    ▼
  7. 将 Raw 材料状态标记为 "compiled"
```

编译后的 Wiki 词条示例：

```
---
type: concept
created_from: raw/articles/xxx.md
created_at: 2026-04-15T10:30:00
related_topics: [[Transformer]], [[Attention Mechanism]]
confidence: 0.92
tags: [LLM, AI]
---

# Transformer 架构

## 定义
Transformer 是一种基于自注意力机制的神经网络架构...

## 详解
[详细解释 200-300 字]

## 关键要点
- 核心是自注意力机制
- 支持并行训练
- 已成为 LLM 的标准架构

## 相关概念
[[注意力机制]] [[位置编码]] [[多头注意力]]

## 来源
- [[文章标题]]
```

3.4 第三步：深度查询（有状态 Wiki 导航）

```
PYTHONPATH=. python scripts/chat.py  # 交互式聊天
```

查询流程详解：

```
用户提问："Transformer 和 RNN 的区别？"
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│ Step 1: 从 index.md 加载相关索引  │
│       匹配包含 "Transformer"    │
│       和 "RNN" 的主题段落       │
└──────────────────────────────────────────────────────────┘
        ▼
┌──────────────────────────────────────────────────────────┐
│ Step 2: 加载匹配的 Wiki 词条      │
```

```
│      读取 wiki/concepts/      │
│      下的完整词条内容         │
└──────────────────────────────┘
          ▼
┌──────────────────────────────┐
│ Step 3: Token 预算管理        │
│      控制在预算范围内         │
│      (默认 100,000 tokens)    │
└──────────────────────────────┘
          ▼
┌──────────────────────────────┐
│ Step 4: 双向链接遍历 (1-hop)  │
│      从已加载词条出发         │
│      找到所有 [[关联]] 词条   │
└──────────────────────────────┘
          ▼
┌──────────────────────────────┐
│ Step 5: 查找已有 Q&A 记录     │
│      复用之前的高质量问答     │
└──────────────────────────────┘
          ▼
┌──────────────────────────────┐
│ Step 6: LLM 综合生成回答      │
│      标注知识边界             │
│      (Wiki 知识 vs 通用知识)  │
└──────────────────────────────┘
          ▼
┌──────────────────────────────┐
│ Step 7: 保存高质量 Q&A        │
│      自动存入 outputs/qa/     │
│      供下次查询复用           │
└──────────────────────────────┘
```

回答示例：

```
### 来自知识库的内容

根据知识库中的 [[Transformer 架构]] 词条...

### 补充说明

知识库尚未收录 RNN 的详细对比，但一般来说...
```

3.5 第四步：健康检查

```
PYTHONPATH=. python scripts/health_check.py --verbose
```

检查项	检测内容	处理方式
断裂链接	[[概念]] 但词条不存在	需要创建缺失的词条
孤岛词条	没有任何入链或出链	需要添加链接或删除
潜在矛盾	LLM 两两比对词条内容	需要人工审核
过期内容	超过 90 天未更新	需要更新或归档
来源缺失	词条没有来源元数据	需要补充引用

3.6 目录结构全景

```
Personal Knowledge System/
│
├── raw/                    # 原始材料（手动输入，不可变）
│   ├── articles/           #  文章 / 博客 / 论文
│   └── notes/              #  临时想法 / 笔记
│
├── wiki/                   # 结构化知识（LLM 编译，持续生长）
│   ├── index.md            #  导航目录（LLM 生成）
│   ├── concepts/           #  概念词条（双向链接节点）
│   └── topics/             #  主题 MOC（Map of Content）
│
├── outputs/                # 高价值产出
│   └── qa/                 #  问答记录
│
├── src/                    # 核心代码
│   ├── compiler/           #  LLM 编译器
│   │   ├── raw_processor.py    #  Raw 材料读写
│   │   ├── wiki_builder.py     #  LLM 编译 + 英译中
│   │   ├── link_extractor.py   #  双向链接分析
│   │   └── index_generator.py  #  索引生成 / 增量更新
│   ├── query/              #  查询引擎
│   │   ├── agent_query.py      #  有状态 Wiki 问答
│   │   └── context_manager.py  #  Token 预算管理
│   ├── maintenance/        #  健康检查
│   │   └── health_checker.py   #  链接 / 矛盾 / 过期检测
│   ├── server/             #  Web 服务
│   │   └── wiki_chat_api.py    #  FastAPI Chat API
│   └── utils/              #  工具
│       ├── config.py           #  配置管理
│       └── model_router.py     #  多模型路由
│
├── config/                 # 配置文件
│   ├── wiki_config.yaml    #  Wiki 配置
│   └── user_profile.yaml   #  用户偏好
│
├── scripts/                # 工具脚本
│   ├── compile_raw.py      #  编译 Raw → Wiki
│   ├── query_wiki.py       #  命令行查询
│   ├── chat.py             #  交互式聊天
│   ├── health_check.py     #  健康检查
│   ├── start_chat_api.py   #  启动 Chat API
│   ├── pdf_to_md.py        #  PDF 转 Markdown
│   └── translate_agentic_book.py #  英译中工具
│
└── .claude/skills/
    └── wiki-cli/           # Claude Code Skill
        └── SKILL.md
```

3.7 技术栈

层面	技术选择	理由
LLM 调用	阿里云百炼 API（Anthropic	支持多模型路由，8+ 模型可选
模型选择	通义 3.6 Plus（默认） G	中文能力强 中文能力强，成本低
存储格式	Markdown 文件 + 双向链接	人类可读、可编辑、Git 版本控制
Web 框架	FastAPI + uvicorn	轻量、高性能、异步

Token 计数	tiktoken	计数准确，支持多种 encoding
项目结构	Python 模块化 compil	清晰的职责分离 高内聚低耦合

四、关键设计决策

4.1 为什么不用向量数据库？

向量数据库（FAISS / Pinecone / Weaviate）的优势是大规模语义检索，但对我们的场景有四个不匹配：

```
向量数据库方案                    Markdown Wiki 方案

┌──────────────────────────┐      ┌──────────────────────────┐
│ ●●●●●                    │      │ [[概念A]] ── [[概念B]]   │
│ ●●●●                     │      │     │          │         │
│ ●●●                      │      │     ▼          ▼         │
│ (孤立点云)               │      │ [[概念C]] ── [[概念D]]   │
│                          │      │ (互联网络)               │
│ 可解释性：差             │      │ 可解释性：完全透明       │
│ 可维护性：无法编辑       │      │ 可维护性：任意编辑       │
└──────────────────────────┘      └──────────────────────────┘
```

维度	向量数据库	Markdown Wiki
规模适配	适合百万级文档	适合几十到几百级
可解释性	差（不知道"为什么返回"）	完全透明（链接即路径）
可维护性	无法直接编辑	任意文本编辑器可改
知识关联	孤立存储	双向链接网络
版本控制	不支持	Git 天然支持

我们的方案：index.md 导航 + 双向链接图。LLM 先通过索引定位相关主题，再遍历链接获取上下文。完全可解释、可编辑、可维护。

4.2 为什么手动输入而不是自动采集？

这是从旧架构踩坑后得出的最重要结论：

```
自动采集流程                      手动输入流程

┌──────────────────────────┐      ┌──────────────────────────┐
│ RSS: 每天 50 篇          │      │ 用户主动保存:            │
│ 有价值: 5 篇 (10%)       │      │ 每天 1-2 篇              │
│ 噪声: 45 篇 (90%)        │      │ 价值率: ~90%             │
│ 价值密度: 极低           │      │ 价值密度: 高             │
└──────────────────────────┘      └──────────────────────────┘
        │                                 │
        ▼                                 ▼
┌──────────────────────────┐      ┌──────────────────────────┐
│ 知识库变成               │      │ 每条知识都经过           │
│ "信息垃圾场"             │      │ 用户思考和筛选           │
└──────────────────────────┘      └──────────────────────────┘
```

- 自动采集 = 低质量噪声灌入：RSS 和爬虫每天灌入 10-50 篇内容，但真正有价值的不到 5%
- 人工筛选 = 高质量输入：用户主动保存的内容，本身就是经过思考的精选
- 用户思考是知识的核心：Raw 材料中的「我的思考」部分，是知识区别于信息的关键

4.3 为什么用 Markdown 而不是数据库？

优势	说明
Obsidian 原生兼容	直接浏览、编辑、可视化知识图谱，零成本接
Git 友好	每次编译和修改都可通过 Git 追踪知识
人类可读	不需要任何工具就能打开阅读，降低使用门槛
LLM 友好	LLM 天然理解 Markdown 格式

4.4 多模型路由的意义

系统支持 8+ 模型，按任务自动路由，并配备完整的容错机制：

```
任务请求
          |
    ┌─────────────────┼─────────────────┐
    |       |       |
    ▼       ▼       ▼
  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
  | 概念提取  | | 翻译   | | 查询回答 |
  |       | |       | |       |
 ▼      ▼▼     ▼▼     ▼
 qwen3.6  qwen3.6-plus  qwen3.6-plus
 -plus   (快速响应)  (综合能力)
  |       |       |
  └──────────────┼──────────────┘
          |
          ▼
   ┌──────────────────────┐
   | 容错机制      |
   |  ┌──────────────┐  |
   |  | 5 次重试  |  |
   |  | 指数退避  |  |
   |  | 自动降级  |──┼── GLM-5 (备用)
   |  | 到 GLM-5  |  |
   |  └──────────────┘  |
   └──────────────────────┘
```

容错保障：即使主模型不可用，服务也不会中断。5 次重试 + 指数退避 + 自动降级到 GLM-5。

五、项目现状

已实现的能力

模块	功能	状态
Raw 材料管理	读取 / 创建 / 状态跟踪	完成
LLM 编译	概念提取 / 词条生成 / 双向链接	完成
英译中翻译	英文内容自动翻译编译	完成
索引生成	LLM 组织 / 增量更新 / 标签降级	完成
链接分析	断裂检测 / 孤岛检测 / 枢纽识别	完成
Wiki 查询	Token 预算 / 链接遍历 / Q&	完成

健康检查	矛盾检测 / 过期检测 / 来源检查	完成
Chat API	FastAPI 服务 / 统计 / 健康	完成
模型路由	多模型 / 自动降级 / 重试	完成
CLI Skill	wiki-cli skill / wik	完成
PDF 转 Markdown	PDF 文档转换	完成

精简成果总结

```
精简前              精简后
┌──────────────────────┐      ┌──────────────────────┐
│ 代码: 9400+行 │ ──── │ 代码: ~5000行 │ -47%
│ 依赖: 42 行   │ ──── │ 依赖: 16 行   │ -62%
│ 模块: 13 个   │ ──── │ 模块: 5 个    │ -62%
│ 脚本: 26 个   │ ──── │ 脚本: 7 个    │ -73%
│ 配置: 6 个    │ ──── │ 配置: 2 个    │ -67%
└──────────────────────┘      └──────────────────────┘
```

仓库地址

https://github.com/Fatshady-Scorpio/Personal-Knowledge-system

六、未来方向

6.1 短期 -- 已规划

功能	说明	优先级
Obsidian 插件集成	在 Obsidian 内直接进行 Wik	高
知识图谱可视化	基于双向链接生成可视化图谱	中
批量 Raw 导入	旧架构知识批量迁移到新系统	中

6.2 中期 -- 探索中

功能	说明
知识蒸馏	当某主题积累足够时，LLM 自动融合知识
知识缺口识别	通过链接分析发现「应该存在但尚未创建」的
多轮对话上下文	支持更复杂的连续追问和上下文保持

6.3 长期愿景 -- 构建「第二大脑」

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  每天阅读、思考、学习的知识                                          │
│        │                                                             │
│        ▼                                                             │
│  被结构化存储为 Wiki 词条                                            │
│        │                                                             │
│        ▼                                                             │
│  通过双向链接自然关联                                                │
│        │                                                             │
```

```
│       ▼                    │
│  遇到问题时，系统基于已有知识给出回答       │
│       │                    │
│       ▼                    │
│  随着时间推移，知识库越来越丰富          │
│       │                    │
│       ▼                    │
│  形成真正的知识复利              │
│       │                    │
│       ▼                    │
│  不是被动的存储，而是主动的知识伙伴        │
│                            │
└──────────────────────────────────────────────────────────────────────────────┘
```

文档版本：1.0

最后更新：2026-04-15

作者：Personal Knowledge System Team